

ARMIS Platform

Architecture Decision Records (ADR)

This document logs significant architectural decisions made during the development of the ARMIS platform. Each ADR captures the context, the decision, alternatives considered, and the current status.

ADR-001: Microservices Architecture

Date	2024-03-10
Status	ACCEPTED
Deciders	Emre Yildirim (Platform Lead), Ahmet Kaya (CTO)

Context

ARMIS needed to support multiple independent teams working on different functional areas (auth, assets, missions) simultaneously. The initial prototype was a monolith and was already showing merge conflict and deployment coupling issues at 8 developers.

Decision

Adopt a microservices architecture with one service per major business domain. Each service owns its database schema. Services communicate via gRPC internally.

Alternatives Considered

- Modular monolith — rejected because deployment coupling was the core problem; modules in a monolith deploy together
- Serverless (AWS Lambda) — rejected due to cold start latency requirements (<200ms p99) and lack of on-premises deployment capability (customer requirement)

Consequences

- Each team can deploy their service independently
- Cross-service queries require API calls, not DB joins — accepted trade-off
- Operational complexity increased: requires service discovery, distributed tracing, and health monitoring

ADR-002: PostgreSQL Per-Service Schema Isolation

Date	2024-03-15
Status	ACCEPTED
Deciders	Emre Yildirim, Zeynep Demir (DB Lead)

Context

With microservices decided, we needed a data isolation strategy. Three options were on the table.

Decision

Use a single PostgreSQL cluster with one schema per service in dev/staging. Use separate PostgreSQL instances per service in production. Flyway manages migrations per service.

Alternatives Considered

- Separate DB per service from day one — rejected for dev/staging due to operational overhead; not practical for a 6-person team
- Shared schema, table-name prefixed — rejected because it provides no isolation and makes accidental cross-service joins possible
- NoSQL (MongoDB) — rejected because mission and asset data is highly relational and benefits from ACID transactions

Consequences

- Dev and staging share one Postgres instance — acceptable risk, reduces infra cost
- Production has full isolation — meets security requirements
- Cross-schema joins at DB level are forbidden by convention; enforced by code review

ADR-003: JWT for Authentication

Date	2024-04-02
Status	ACCEPTED
Deciders	Emre Yildirim, Burak Sen (Security Lead)

Context

The platform needed a stateless authentication mechanism compatible with horizontal scaling. The API Gateway needed to validate tokens without a database call on every request.

Decision

Use signed JWTs (HS256) for access tokens. Access token TTL: 8 hours. Refresh token TTL: 7 days, stored in Redis for revocation support.

Alternatives Considered

- Session-based auth (server-side sessions) — rejected because it requires sticky sessions or a shared session store, complicating horizontal scaling
- OAuth2 / OpenID Connect with external IdP — not yet available; planned for Q3 2026 (ADR-006 pending)
- RS256 asymmetric JWT — considered more secure but adds key management overhead; can be migrated to later without breaking clients

Consequences

- Tokens cannot be individually revoked before expiry — mitigated by short TTL (8h) and refresh token revocation in Redis
- Role changes take effect on next token issuance — operators must re-login after role changes
- JWT_SECRET must be kept in sync across all services — a rotation procedure is documented in [runbooks/jwt-rotation.md](#)

ADR-004: Async Report Generation

Date	2024-07-20
Status	ACCEPTED — REVISION PENDING
Deciders	Emre Yildirim, Fatma Ozturk (report-service owner)

Context

PDF and Excel report generation using Jasper Reports can take 5-30 seconds depending on data volume. Synchronous generation caused request timeouts (30s gateway limit) for large mission reports.

Decision

Make report generation asynchronous. Client submits a request, receives a reportId, and polls for completion. Generated files are stored in S3. Download via pre-signed URL.

Current Problem (Revision Pending)

The current implementation uses Spring's `@Async` with a shared thread pool. Under high concurrency (10+ simultaneous report requests), Jasper rendering saturates the thread pool and causes latency spikes across other report-service endpoints. A revision (ADR-004b) is being drafted to introduce a dedicated worker pool with a configurable concurrency limit.

Alternatives Considered

- Dedicated report worker service — under consideration for ADR-004b
- Client-side report generation (export data, client renders) — rejected because some reports require server-side aggregation not exposed via API

ADR-005: RabbitMQ for Event-Driven Notifications

Date	2024-08-05
Status	ACCEPTED
Deciders	Emre Yildirim, Canan Arslan (DevOps)

Context

mission-service needed to trigger notifications when mission status changed, without being coupled to notification-service. Direct HTTP calls would create a runtime dependency and could cause mission status updates to fail if notification-service was down.

Decision

Use RabbitMQ with a topic exchange (armis.events). mission-service publishes events; notification-service consumes them. Services are decoupled — mission-service does not know notification-service exists.

Alternatives Considered

- Direct REST call from mission-service to notification-service — rejected due to coupling and availability dependency
- Apache Kafka — evaluated but rejected as operationally heavier than needed for current message volume (<1000 events/day)
- AWS SNS/SQS — rejected due to on-premises deployment requirement

Consequences

- Notifications are eventually consistent — a mission status change may take 1-3 seconds to result in a push notification
- Dead letter queue (armis.notifications.dlq) captures failed deliveries — monitored via RabbitMQ Management UI
- If RabbitMQ is down, mission status updates still succeed — notifications are lost (acceptable per product requirement)